

(1) main.c 中: main, x, z 是强符号
y 是弱符号.

proc1.c 中: proc1 是强符号.
x 是弱符号.

(2) 1.5 的十六进制表示是 0X3FF8 000000000000
执行结果: x = 0, z = 0

调用前: &x: 01010000
&z: 0200

调用后: &x: 00000000
&z: 0000

改为 short y = 1, z = 2; 后

执行结果

x = 0, z = 16376.

(3) static double x;
void proc1() {
 x = 1.5;
}

二. 这是由于符号解析冲突, m1.c 中的 main 是强符号.
而 m2.c 中的 main 是弱符号. 链接器最终选择了
强符号的地址. 因此, m2.c 中的代码在读取 main
数组时, 实际上读取 main 函数的机器代码.

三. (1) gcc p.o libx.a liby.a

(2) gcc p.o libx.a liby.a libx.a

(3) gcc p.o libx.a liby.a libx.a libz.a

